Stage 2 Implementation Report

AI-powered Study Planner | EECS 3311 | Rafat Kabir
Progress draft: F01–F02, November 1, 2026.

1. Overview and status

The application helps students organize courses and, in later features, generate AI study resources. Students can create courses, open a course workspace, and upload learning materials through a Vaadin GUI or Java CLI. A separate Spring Boot backend stores courses and materials in H2.

F01 and F02 ingestion are implemented. Editing, deletion, and AI resource generation remain pending. Authentication supports one locally configured account and does not count as a project feature.

2. Feature-to-implementation mapping

F01: Course Creation. Deterministic. References: Stage 1 Task 1.2 F01, UC-01, SD01, and Task 4 F01. Students enter a name, description, start date, and end date. The system validates the input, saves an owned course, and displays it on the dashboard. Video: pending.

Part	Files, responsibilities, and key methods
GUI	`frontend/.../views/MainView.java`: `showSignIn`, `showDashboard`, `openCourseForm`, `refreshCourses`, and `renderCourses` handle sign-in, forms, errors, and course cards. `themes/study-planner/styles.css` styles the UI.
API client	`frontend/.../course/CourseApi.java`: `createCourse`/`viewCourses` call the backend. `CourseApiFactory.connect` creates a client for each signed-in view. Frontend `Course` and `CreateCourseRequest` records carry API data.
REST API	`backend/.../course/CourseController.java`: `createCourse`, `viewCourses`, and `getCourse` expose POST/GET `/api/courses` and GET `/api/courses/{id}`.
Application and domain	`CourseService` coordinates creation and retrieval. `User.createCourse/viewCourses` create and retrieve owned courses. Backend `Course` validates domain data; `CreateCourseRequest` validates API fields.
Storage	`CourseRepository` defines storage operations. `JdbcCourseRepository.save/findByOwner/findByIdAndOwner` use SQL. `schema.sql` defines the table. H2 files persist under `backend/data/`.

Authentication and errors	`SecurityConfiguration` verifies credentials; `CsrfController.token` supports protected writes. `ApiExceptionHandler` handles invalid requests and storage failures. The authenticated student determines course ownership.
CLI	`cli/StudyPlannerCli.java`: `main/run` provide `create`, `list`, and `help` using the same authenticated API.

Flow: form or CLI → CourseController → CourseService → User.createCourse → CourseRepository → H2. The GUI then reloads the saved course list. Failed submissions show errors and preserve form input.

All four fields are required. Names allow 120 characters; descriptions allow 2000. Text is trimmed, and end dates must be on or after start dates. Same-day courses and duplicate names are allowed. Course queries are restricted to their owner.

F02: Upload Learning Materials. References: Stage 1 Task 1.2 F02, UC-02, SD01, and Task 4 F02. Open an owned course, select files, and view the saved materials and extracted text. Upload/validation/text extraction are deterministic; the planned AI processing belongs to later features. Video: pending.

Part	Implementation and key methods
GUI / client	`MainView.openCourse` opens `CourseMaterialsPanel`; upload, `refreshMaterials`, and `viewText` show results and errors. Each file offers View text and Download original. `CourseApi.uploadMaterial/listMaterials/materialContent/downloadMaterial` call the authenticated API.
API / service	`CourseController.uploadMaterial/listMaterials/materialContent/downloadMaterial` delegate to `MaterialService`, which verifies course ownership before access.
Domain / parsing	`Course.addMaterial` creates `LearningMaterial`; `getContent` exposes extracted text. `MaterialParser.parse` checks filenames and parses PDF, DOCX, or UTF-8 TXT using PDFBox, POI, or UTF-8 decoding.
Storage / CLI	`MaterialRepository` and `JdbcMaterialRepository` save metadata, original BLOB, and extracted text together. `StudyPlannerCli` adds `upload`, `materials`, and `content`.

Rules: up to 10 files, 10 MiB each; unsupported, empty, corrupt, encrypted, or text-free files are rejected. No OCR is provided. Text is limited to 1,000,000 characters, PDF to 500 pages, and expanded DOCX to 20 MiB/1000 entries. API batches roll back on any failure; GUI files upload independently. Originals and text survive restarts.

Remaining Stage 1 features are **not implemented**:

ID	Feature	ID	Feature
F03	Document Summarization	—	—

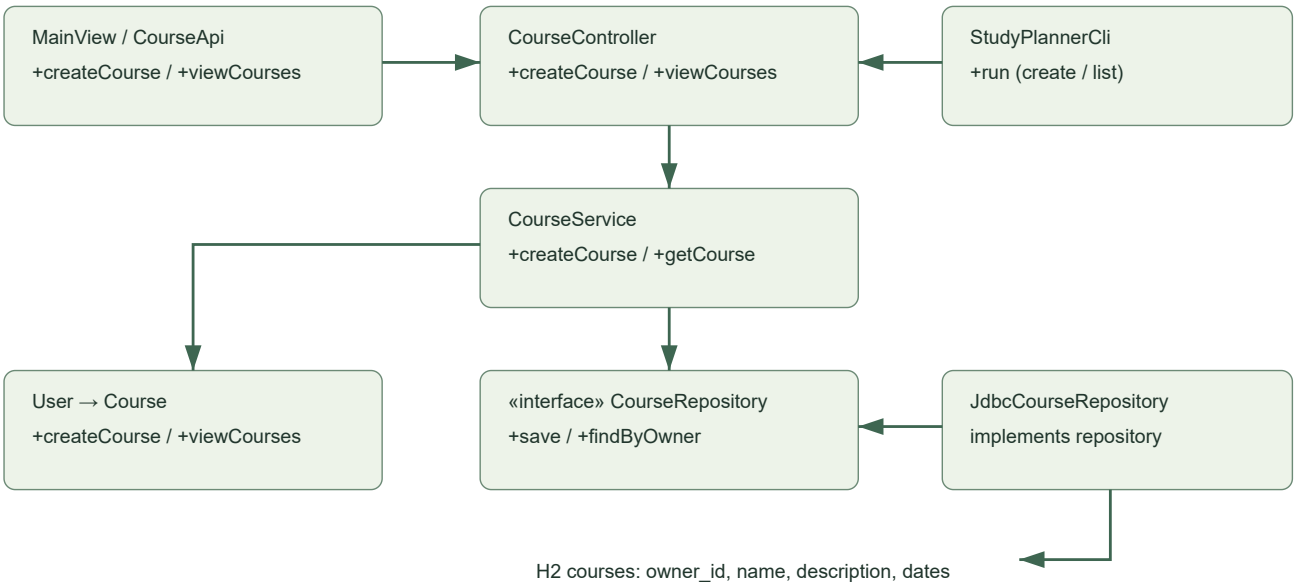
F04	Concept Extraction	F05	Question Answering Over Materials
F06	Personalized Study-Plan Generation	F07	Flashcard Generation
F08	Quiz Generation	F09	Automatic Quiz Grading
F10	Calendar	UC-11	Maintain Study Profile (supports F06)

3. Design patterns and diagrams

The five Stage 1 patterns remain pending: Strategy for grading, Observer for calendar updates, Facade for AI operations, Singleton for AIEngine, and Factory for study resources. F01–F02 do not implement these patterns. Spring singleton beans and `CourseApiFactory` are not counted toward them.

F01 adds a Repository abstraction: `CourseService` depends on `CourseRepository`, implemented by `JdbcCourseRepository`. This keeps SQL separate from application logic; it does not replace a planned pattern.

Current F01 class relationships:



F02 adds CourseController → MaterialService → Course.addMaterial → LearningMaterial → MaterialRepository/JdbcMaterialRepository. MaterialParser validates and extracts text. Updated diagrams: `documentation/diagrams/stage2-f01.md` and `stage2-f02.md`. Original Stage 1 diagrams are preserved.

4. Changes and rationale

Stage 1 design	Implementation change	Reason and impact

UI → CourseController → User → Course (F01/UC-01/SD01).	Added CourseApi, CourseService, CourseRepository, JDBC, and H2.	Connects the separate apps and saves courses across restarts. The creation sequence is updated.
User owns a collection of courses.	User creates courses; repository queries retrieve them by owner UUID derived from username.	Supports persistent ownership. Changing the username changes the owner identity; account persistence is future work.
Authenticated student required; authentication mechanism unspecified.	One configured account, Spring Security, HTTP Basic, and CSRF protection.	Enforces the precondition. Credentials stay in ignored local configuration. Remote deployment requires HTTPS.
Four course fields; detailed limits unspecified.	Required fields, text limits, trimming, and ordered dates.	Rejects invalid courses consistently. Editing/deletion remain pending.
GUI and CLI required.	Java CLI calls the same REST API as the GUI.	Reuses validation and storage. Future agent features still need CLI support.
F02/UC-02/SD01: CourseController → Course.addMaterial → LearningMaterial; file rules unspecified.	Added course navigation, MaterialService, parser, repository, text previews, and browser downloads of originals. PDF/DOCX/TXT, size and extraction limits apply.	Students can read extracted text or save the complete document, preserving formatting and images. Durable BLOB/CLOB storage keeps originals and text consistent. No AI or OCR is claimed.

No original feature or planned pattern has been removed or replaced.

5. AI tools and collaboration

Tool: OpenAI Codex, a GPT-6-based coding agent; exact runtime model ID unavailable. Used to review requirements and diagrams, implement F01–F02, write tests, and maintain this report.

Interaction: the student requested F01, a shorter report, then F02. Codex implemented the course and material flows and their verification while retaining the concise report structure.

Tokens and monetary cost: unavailable in this session; the student must add platform usage and billing totals before submission. Human code review and understanding remain required. F01–F02 ingestion make no LLM calls.

6. Development and verification

Twelve backend tests and the production frontend build pass. Tests cover courses, real PDF/DOCX/TXT uploads, parsing, text/original retrieval, invalid files, limits, batch rollback, authentication, CSRF, and owner isolation. Chrome/CLI checks pass for F01 creation and F02 multi-file upload, text preview, original downloads (exact names/bytes), file errors, course isolation, keyboard/mobile access, reload, backend failure, and restart persistence.

Checks: `backend/mvnw.cmd clean verify`, frontend `mvnw.cmd package -Pproduction`, and `scripts/verify-f01.ps1`. Runtime checks use isolated test data and stop their own servers. A missing BCrypt encoding prefix was found and corrected during testing.

F02 runtime checks use `scripts/verify-f01.ps1 -IncludeF02`. Real-client checks caught a text-preview Accept-header mismatch and a parallel-upload CSRF session race. Correct response headers and serialized token/write requests fixed them.

Toronto reporting period	Minimum code commits	Observed local evidence
October 20–26, 2026	5	No qualifying local commits found; remote evidence needs checking.
October 27–November 2, 2026	5	a0db921: starter; 42853d5: F01. Push status not verified.
November 3–9, 2026	5	Pending.

Local credentials and database files are ignored by Git. `backend/config.local.properties.example` provides placeholders; setup instructions are in the README. F01 requires no API key.

Before the November 10 submission: complete the remaining features, five patterns, LLM/agent behavior, agent CLI, full-system UML, videos, cost records, and commit evidence. Check any additional baseline assignment instructions; only the supplied Stage 1 report and Stage 2 instructions were reviewed.

7. Demonstration index

Feature	Video status	Demonstration scope
F01 Course Creation	Recording and accessible link/timestamp pending.	Identify F01; show GUI and CLI creation, input errors, dashboard output, and persistence. Explain that F01 is deterministic. Keep credentials hidden.
F02 Upload Learning Materials	Recording and link/timestamp pending.	Show course navigation, GUI/CLI upload, saved files, preview, original downloads, invalid/type/size errors, and persistence. Identify deterministic extraction and pending AI processing.
F03–F10 and UC-11	Pending implementation.	Add a labeled video segment for each implemented feature.

Automated browser checks do not replace demonstration videos. Update this source and regenerate `stage2-report.pdf` as features are completed.